

What is an API?

- ↳ Application Programming Interface
- ↳ An API provides end-points (functions)
- ↳ In case of OS, it is called system calls

↳ System calls are:

- ↳ OS code

- ↳ Runs at a higher "Privilege level"

- ↳ Can do things user code can't i.e access hardware

POSIX:

- ↳ Portable OS Interface Standard

- ↳ API standard

- ↳ Ensures compatibility across OS

- ↳ Libraries provide an easy to use interface to system calls

- ↳ C has libc library

- ↳ printf calls write system call

- ↳ Some of them are:

1. fork()

- ↳ creates a process

2. exec()

- ↳ executes an .exe

3. exit()

- ↳ Terminates a process

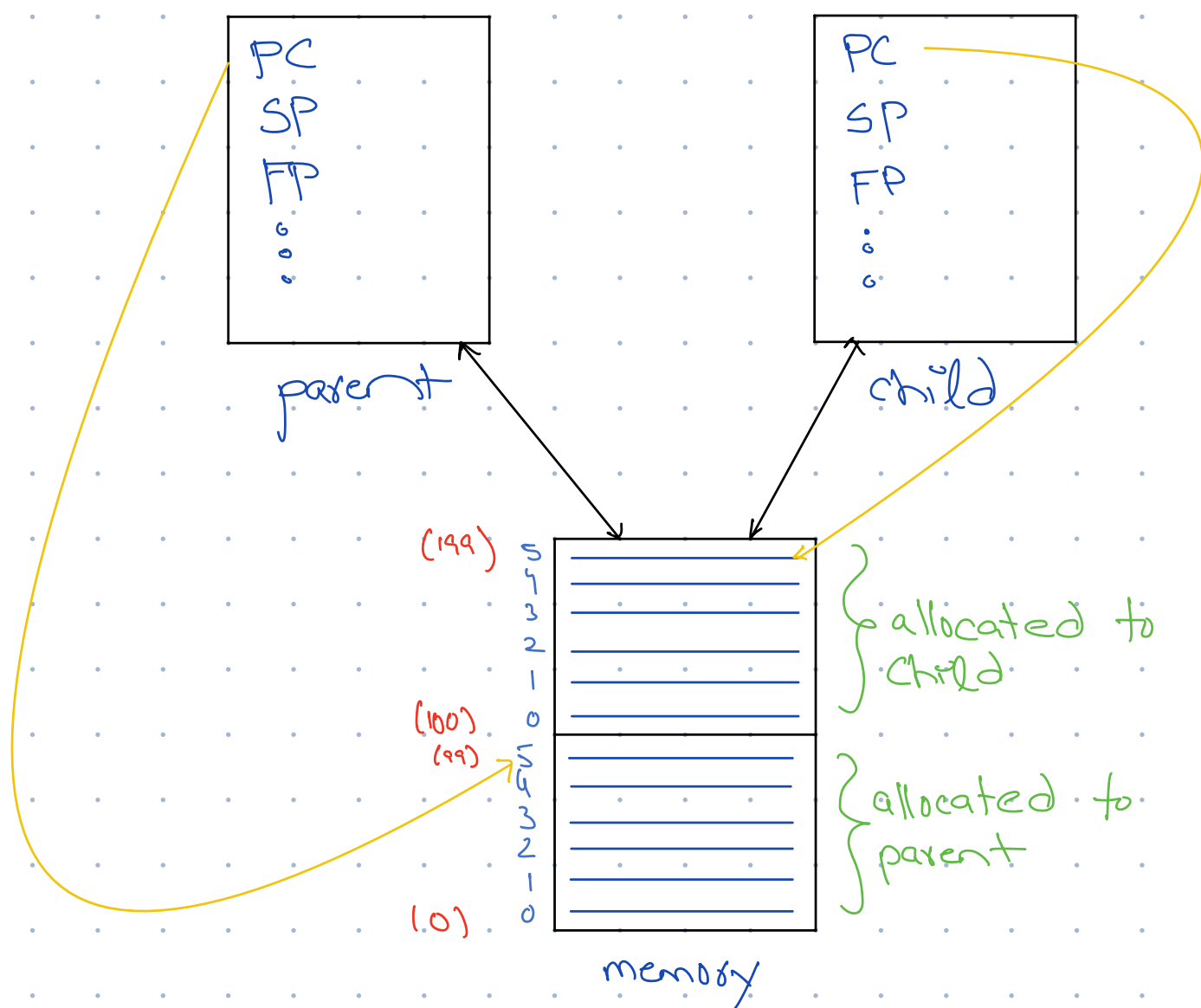
4. wait()

- ↳ wait for another process to terminate.

CRUX: How to create & control processes
What interfaces should OS present for process creation & control?

The `fork()` system call

- ↳ It is used to create a new process.
- ↳ The newly created process is an exact copy of the calling process.
- ↳ Although, the address space is different i.e. processes don't share memory but the PC, SP, FP point to the same point as in parent process.



- ↳ The `fork()`, unlike a normal function, has two return calls.
- ↳ To the child process, it returns 0.
- ↳ To the parent process, it returns the pid of the child process.

↳ We can't know which process will run first.

The `wait()` system call

- ↳ Sometimes, it is useful to wait for a child process to finish what it has been doing.

- ↳ This done by using `wait()` system call.
- ↳ This always us to control which process runs first.

The `exec()` system call

↳ This is used to make the child process run something else.

↳ `Exec`, given the name of an `.exe` & some arguments, it:

1. Loads code & static data from that `.exe` & overwrites it's current code segment (and current static data).
2. The stack, heap & other parts of memory space of the new process are initialized.

↳ `exec()` never returns when successful.

Process termination:

↳ A Process can terminates either:

↳ voluntarily by calling `exit()` or by the OS

↳ Parent could `wait()` for child process to terminate

↳ If parent process terminates first, then child is orphan process.

↳ In this case `init` process becomes the parent process.

Why? Motivating the API

↳ Why do we separate creation of a process into `fork()` and `exec()`?

↳ Because it allows the shell run code after the call to `fork()` but before the call to `exec()`.

↳ The shell (`bash`, `zsh`) shows you a prompt & waits for you to type a command (name of an `.exe`) + args

↳ The shell finds the where `.exe` is, it then calls `fork()` to create a new child process to run the command, then runs the command using `exec` & waits for it to complete by using `wait()`.

↳ This separation allows:

```
wc p3.c > newfile.txt
```

↳ The output of `wc` is written to the `.txt` file.

↳ This works because of how OS manages file descriptors.

↳ UNIX opens below files by default, the OS assigns each an integer.

- `STDIN_FILENO` (0)
- `STDOUT_FILENO` (1)
- `STDERR_FILENO` (2)

↳ When you close `STDOUT_FILENO` and open a new file, the OS will assign it the lowest available slot (which is 1 here)

↳ So, any `printf` call will be re-directed to the newly opened file & not the `stdout` & thus not printed to the terminal.

↳ Similarly, unix pipes are implemented in a similar way, but with `pipe()` system call.

↳ In this case, the output of one process is connected to an in-kernel pipe (i.e. queue); thus, output of one process can seamlessly be used as input to another.

Other parts of the API:

• `kill()` system call is used to send signals to a process, including directives to sleep, die ect.